소프트웨어공학 CHP 1-5

시험 직전 압축 암기본

- 범위: 노션 소프트웨어공학 1장부터 5장
- 구성: 핵심어 · 비교 · 공식 · 시험 포인트
- 이해 장치: 치킨 주문 시스템
- 문체: 문장 설명보다 짧은 구절 중심

정리 기준: Notion 소프트웨어공학 1장-5장

형식: 핵심 구절 · 비교표 · 공식 · 시험 포인트

예시: 치킨 주문 시스템

전체 흐름

다섯 장의 연결

장	핵심 질문	치킨집으로 기억하기
1장 소개	왜 소프트웨어공학이 필요한가	레시피만이 아니라 주문·조리·검수·운영 전체
2장 프로세스와 방법론	어떤 순서와 방식으로 개발하는가	한 번에 개점할지, 작은 기능부터 반복 출시할지
3장 프로젝트 관리와 계획	일정·비용·인력·위험을 어떻게 관리하는가	오픈 일정, 주방 인력, 예산, 배달 지연 위험 관리
4장 요구 분석	사용자가 원하는 것을 어떻게 정확히 찾는가	메뉴, 주문 방식, 응답 시간, 보안 조건 확인
5장 요구 모델링	요구를 어떤 그림과 구조로 표현하는가	고객·주문·결제의 관계와 주문 흐름을 UML로 표현

한 줄 흐름

- 아이디어 확인
- 요구 추출·분석
- 요구 모델링
- 설계
- 구현
- 테스트·통합
- 유지보수

시험에서 먼저 구분할 것

- 프로세스: 무엇을 어떤 순서로 수행하는가
- 방법론: 각 작업을 어떻게 수행하는가
- 요구 분석: 무엇이 필요한지 찾아 정리
- 요구 모델링: 합의한 요구를 구조와 그림으로 표현
- 프로젝트 관리: 범위·일정·비용·인력·위험 통제

1장 소개

소프트웨어

- 프로그램
- 개발·운용·보수에 필요한 정보 일체
- 무형의 생산물
- 구조가 코드 안에 숨음

소프트웨어 특징 - 복순변비

- 복잡성: 구성 요소와 상호작용이 많음
- 순응성: 법·업무·하드웨어·조직 환경에 맞춰야 함
- 변경성: 요구와 환경에 따라 계속 수정
- 비가시성: 물리적 형태가 없어 구조 파악이 어려움

소프트웨어 분류

유형	핵심	특징	예
주문형	특정 고객 요구에 맞춤 제작	카피 수 적음 · 개발 인력 큼	기업 전용 주문 관리
패키지	다수 고객에게 판매	카피 수 많음 · 범용 기능	문서 작성 · POS · 회계
임베디드	다른 시스템 내부에 탑재	자원·성능 제약 큼	자동차 제어 · 가전

프로그래밍과 소프트웨어공학

프로그래밍	소프트웨어공학
개인 작업 중심	팀 프로젝트 중심
코드 작성 중심	개발 전체 단계 포함
시행착오와 수정	공학 원리와 계획 적용
실행되는 프로그램이 목표	타당성·품질·미래 사용까지 고려
언어·알고리즘·자료구조	분석·설계·구현·테스트·관리 능력

치킨집 연결

- 프로그래밍: 주문 버튼 하나 구현
- 소프트웨어공학: 메뉴 요구·결제 설계·배달 연동·테스트·운영까지 관리

소프트웨어 개발의 기본 활동

- 명세화
  - 작동 방식·성능·제약 협의
- 구현
  - 설계 후 프로그래밍 언어로 변환

- 검증
  - 고객 의도와 결과 일치 확인
- 유지보수
  - 사용 중 발견된 결함 수정·기능 변경

개발 작업이 어려운 이유

- 명세화 어려움: 고객 의도 설명·이해의 차이
- 재사용 어려움: 프로젝트마다 문제와 환경이 다름
- 예측 어려움: 사람·기술·요구·조직 변수
- 유지보수 어려움: 구조 이해와 고장 위치 파악 난해
- 고품질 어려움: 관련 요소와 상호작용이 많음

소프트웨어 위기

- 요구·복잡도·난이도 증가
- 인력·방법·도구의 발전 부족
- 결과
  - 개발 지연
  - 예산 초과
  - 낮은 품질
  - 유지보수 곤란
  - 반복 재작업

소프트웨어공학

- 개발·운영·유지보수·소멸에 대한 체계적 접근
- 반복 가능한 방법 사용
- 목표
  - 복잡도 감소
  - 비용 최소화
  - 개발 기간 단축
  - 대규모 프로젝트 관리
  - 고품질·효율성 확보
- 최종 문장
  - 품질 좋은 소프트웨어
  - 최소 비용
  - 계획된 일정

소프트웨어공학의 세 주제

단계적 프로세스 - 생산성

- 정해진 순서로 작업

- 생명주기
  - 계획 → 분석 → 설계 → 구현 → 테스트·통합 → 유지보수
- 프로세스 부재
  - 결과 예측 어려움
  - 품질 저하 가능성 증가

품질 보증 - 품질

- 검토: 개발 주기와 절차 준수 확인
- 확인: 산출물이 품질 수준에 맞는지 확인
- 테스팅: 구현 결과가 예상 결과를 내는지 확인

프로젝트 관리 - 비용·시간

- 프로젝트 계획: 범위·타당성·방법·일정·조직
- 자원 관리: 필요한 자원 산정·할당
- 리스크 관리: 위험 예측·분석·대비
- 수행과 모니터링: 진척 확인·조정·조처

발전 흐름

- 1960년대: 프로그래밍 중심 · 개인 기술
- 1970년대: 구조적 기법 · 설계 · 모듈화 · 폭포수
- 1980년대: 객체지향 분석·설계 · 테스트 · 비용 예측
- 1990년대: 분산 시스템 · UML · 디자인 패턴 · 성숙도 · 재사용
- 2000년대: 애자일
- 2010년대: 클라우드 시스템 기술

1장 암기

- 특징: 복순변비
- 핵심 활동: 명세화 · 구현 · 검증 · 유지보수
- 핵심 축: 프로세스 · 품질 보증 · 프로젝트 관리

2장 프로세스와 방법론

소프트웨어 생명주기

- 요구 분석 → 설계 → 구현 → 테스팅 → 유지보수

프로세스

- 일을 수행하는 단계·작업의 집합
- 작업 사이 순서 제약 존재
- 목표: 높은 품질·생산성
- 프로세스 없는 개발
  - Code-and-fix
  - 코딩 → 만족할 때까지 수정

프로세스 명세와 프로세스 모델

구분	프로세스 명세	프로세스 모델
범위	특정 프로젝트	일반적인 표준 절차
내용	실제 작업과 수행 순서	단계·순서·조건·제약
특징	프로젝트에 맞게 커스터마이징	여러 프로젝트에 적용 가능한 틀

좋은 프로세스

- 예측 가능성: 비용·작업·품질 예측
- 테스트·유지보수 용이성: 설계·코드 품질과 비용에 영향
- 변경 용이성: 요구·조직·환경 변화 수용
- 결함 제거 용이성: 오류 발견 즉시 해당 단계에서 수정

설계 프로세스 정의

- 목적: 기대 성과와 수행 이유
- 작업 방법: 해야 할 구체 작업
- 성과: 문서·코드·프로토타입
- 순서
  - 설계 준비
  - 요소별 설계 확립
  - 설계 대안 평가
  - 설계 변경·버전 관리
- 포함 내용: 요소 특징 · 요구 할당 · 인터페이스 · 대안 평가

프로세스와 품질

- 제품 품질 ← 프로세스 능력의 영향
- 품질 평가 모델 → 프로세스 품질 중심

- 관리 프로세스
  - 계획: 비용·일정·중간 점검
  - 모니터링·조절: 진척 데이터 수집 → 분석 → 조정

폭포수 모델

- 선형적 진행
- 단계 간 중복·상호작용 적음
- 다음 단계 전 산출물 점검
- 장점
  - 단순한 절차
  - 산출물 명확
  - 관리 용이
  - 분석·설계 시간 확보
- 단점
  - 불필요한 문서 가능
  - 변경 수용 어려움
  - 구현 완료 후 테스트 시작
- 적합
  - 요구가 안정적
  - 크고 오래 지속되는 시스템
  - 하드웨어 연동·공정 제어

V 모델

- 폭포수 확장
- 개발 단계와 테스트 단계 대응
- 테스트 수준
  - 단위 테스트: 개별 모듈
  - 통합 테스트: 모듈 인터페이스
  - 시스템 테스트: 전체 시스템
  - 인수 테스트: 사용자 요구 반영

프로토타이핑 모델

- 실험용 시스템 제작 → 사용자 평가·피드백
- 목적
  - 요구 추출용: 확인 후 폐기
  - 실현 가능성 확인: 개발 과정에 활용
- 장점
  - 사용자 의견 반영
  - 요구 정확도 향상

- 개발자·사용자 공통 참조물

- 단점
  - 완제품으로 오해
  - 기대 과잉
  - 중간 산출물·진척 관리 어려움
- 적합
  - 요구 불투명
  - 신기술 실험
  - 실현 가능성 불확실

점증적과 진화적

점증적	진화적
전체 요구를 먼저 명세	초기 모든 요구 미확정
기능별 서브시스템으로 분할	반복 경험으로 요구·제품 발전
기능 묶음 단위 릴리스	더 완성도 높은 버전 반복 릴리스

- 공통 장점: 빠른 사용 · 빠른 피드백 · 시장 조기 진입
- 공통 단점: 관리 복잡 · 종료 모호 · 반복 실패 위험

나선형 모델

- 점증적 개발 + 위험 분석
- 반복 순환
  - 목표·방법·제약 결정
  - 위험 분석·해결
  - 개발·평가
  - 다음 단계 계획
- 장점
  - 대규모·고위험 프로젝트
  - 반복 테스트와 피드백
  - 다음 사이클에서 누락 보완
- 단점
  - 관리 복잡
  - 위험 분석 역량에 크게 의존
  - 잘못된 분석의 피해 큼
- 적합
  - 재정·기술 위험 큼
  - 요구·아키텍처 이해 부족

Unified Process - 네 단계

- 도입 Inception
  - 간단한 유즈케이스·구조·계획
- 정교화 Elaboration
  - 주요 유즈케이스
  - 아키텍처 설계
- 구축 Construction
  - 남은 기능 구현·통합
  - 점증적 설치
- 전환 Transition
  - 배치·교육
  - 베타 테스트·결함 수정·개선

Unified Process - Disciplines

- 요구: 시스템이 해야 할 일
- 분석·설계: 구조와 동작
- 구현: 코드 작성·통합
- 테스트: 품질 검증
- 배포: 사용자 환경 설치·유지
- 프로젝트 관리: 일정·리스크·자원
- 환경: 개발 도구·프로세스 지원

Unified Process 평가

- 장점: 알려진 방법론 · 요구 변경 리스크 대응 · 재사용 · 통합 노력 감소
- 단점: 복잡한 프로세스 · 학습과 적용 어려움 · 협업 가이드 부족

애자일 모델

- 2-6주 짧은 주기 반복
- 실행 가능한 소프트웨어를 단계적으로 완성
- 애자일 선언 핵심
  - 문서보다 사람과 커뮤니케이션
  - 문서보다 작동하는 소프트웨어
  - 계약 고정보다 고객 협력
  - 계획 고수보다 변화 대응
- 운영 특징
  - 우선순위 높은 기능 먼저
  - 품질·비용·일정을 의식하며 기능 범위 조정
  - 단계별 분업보다 협력 팀

- 장점
  - 빠른 배포·피드백
  - 문제·결함 조기 발견
  - 불필요한 작업 감소
- 단점
  - 지속적 상호작용 필요
  - 신규 인력 적응 어려움
  - 종료·결산 모호
  - 전체 UX·아키텍처 약화 가능

익스트림 프로그래밍 XP

- 사용자 스토리: 한 문장의 기능 요구
- 매일 빌드·통합: 항상 실행 가능한 상태
- 테스트 주도 개발 TDD: 테스트 먼저, 코드 나중
- 페어 프로그래밍: 작성자 + 검토자

크리스탈

- 사람 중심 방법론
- 기술·재능·의사소통 중시
- 규모·중요도·일정에 따라 방법 조정

스크럼 역할

- 프로덕트 오너: 요구·우선순위·제품 가치
- 개발 팀: 3-10명 · 구현 책임
- 스크럼 마스터: 리더가 아니라 진행 조력자

스크럼 이벤트

- 스프린트 계획: 이번 주기의 구현 범위
- 일일 스크럼: 15분 이하 진행 확인·도움 요청
- 스프린트 리뷰: 결과 데모·백로그 조정
- 스프린트 회고: 과정·교훈·개선점

스크럼 산출물

- 프로덕트 백로그: 전체 요구·스토리 우선순위
- 스프린트 백로그: 이번 스프린트 작업
- 동작하는 소프트웨어: 배포 가능한 누적 결과
- 번다운 차트: 남은 작업 감소 추적

프로세스와 방법론

- 프로세스

- 어떤 작업을 어떤 순서로 할지

• 방법론

- 각 작업을 어떻게 수행할지
- 단계·입력·산출물·표현 방법 명시

• 패러다임

- 소프트웨어를 바라보는 관점

구조적 방법론

• 1970년대 제안 · 1980년대 활발

• 실세계 문제를 프로세스 관점으로 모델링

• 자료 흐름도 DFD

- 입력 자료 → 처리 → 출력 자료

정보공학 방법론

• 1990년대 제안

• 기업 전체 조망·데이터 통합

• 기업 중심

• 전략적 시스템 계획

• 데이터 중심

• 분할과 정복

• 공학적 접근

• 사용자 적극 참여

객체지향 방법론

• 실세계를 객체 관점으로 이해

• 객체 사이 상호작용 모델링

• 클래스 단위 모듈화

• 캡슐화

• 대규모 복잡 시스템에 적합

• OMT · Booch · 유즈케이스 접근법

• UML로 통합

2장 비교 암기

• 폭포수: 순차 · 안정된 요구

• V: 개발 단계와 테스트 대응

• 프로토타입: 요구 확인

• 점증적: 기능 분할 후 순차 릴리스

• 진화적: 요구와 제품을 반복 발전

• 나선형: 위험 분석 중심 반복

- Unified Process: 도입-정교화-구축-전환
- 애자일: 짧은 주기 · 변화 대응 · 작동 소프트웨어

3장 프로젝트 관리와 계획

프로젝트 관리

- 목적
  - 자원·인력·비용·재료·기술의 효과적 사용
  - 프로젝트 목표 달성
- 어려움
  - 개발 대상 비가시성
  - 빠른 기술 변화
  - 조직마다 다른 프로세스
- 관리 활동
  - 계획
  - 조직
  - 모니터링
  - 조정

계획 부재의 결과

- 불확실성 증가
- 일정 지연
- 비용 초과
- 낮은 품질
- 높은 유지보수 비용
- 위험 확대
- 프로젝트 실패

개발 계획서 핵심

- 문제 정의
- 소작업 정의·순서
- 일정 예측
- 비용 예측
- 위험 분석
- 이해관계자에게 제공할 정보
  - 필요 비용·자원
  - 개발 일정
  - 위험 요소

계획 시 주의

- 시스템 이해

- 변경 가능성 고려
- 현실적·구체적 계획
- 득실 관계 비교
- 기술 조건 확인

프로젝트 시작

- 목표 설정
- 가치 평가
- 리스크 이해
- 가치 평가 방법
  - 투자 회수 기간
  - ROI: 총비용 대비 연간 평균 이익률
  - 순현재가치 NPV: 현재 투자금과 미래 수익의 현재가치 비교
  - 평가표: 기술·품질·시간·인력 점수화
  - SWOT: 내부 강점·약점 + 외부 기회·위협

타당성 분석 항목

- SOW: 프로젝트가 성취할 일
- 비즈니스 목표·가치
- 예산
- 대략 일정
- 리스크
- 구축·구매 등의 대안
- 가치 평가 결과

프로젝트 목표와 범위

- 목표·요구
- 기본 작업 WBS
- 산출물·승인 조건
- 가정·제약 조건
- 점검 일정
- 범위 결정
  - 넓으면 일정·비용·리스크 증가
  - 작은 서브시스템으로 분할 가능

WBS

- Work Breakdown Structure
- 목표 달성에 필요한 작업의 계층적 분할
- 리프 노드 = 실제 수행 작업

- 일정·비용·자원 계획의 기초

스케줄링

- WBS를 일정으로 변환
- 순서
  - 작업 의존 관계 파악
  - CPM으로 여유 시간 계산
  - 자원 할당

CPM 네트워크

- 가중치 그래프
- 노드: 작업
- 간선: 선행 → 후행 의존 관계
- 노드 위 기간 표시
- Critical Path
  - 가장 긴 경로
  - 프로젝트 최소 완료 시간 결정
  - 경로 작업 하나 지연 → 전체 프로젝트 지연
- 여유 시간
  - TS = TL - TE
  - TL: 가장 늦게 시작 가능한 시간
  - TE: 가장 일찍 시작 가능한 시간

간트차트

- 작업별 시작·종료
- 수평 막대
- 계획과 실제 진행 표시

자원 할당

- 인력: 담당자·투입률
- 장비: 도구·하드웨어·소프트웨어
- 재료: 소모품·자료

치킨집 연결

- WBS: 메뉴 등록 · 결제 연동 · 배달 연동 · 테스트
- CPM: 결제 연동 지연이 전체 오픈을 늦추는지 확인
- 간트차트: 각 작업의 시작·종료를 달력 위에 표시

애자일 계획

- 반복 주기 약 3주

- 유즈케이스·스토리 단위 배정
- 스토리별 정보
  - 점수
  - 우선순위
  - 선행 스토리
- 원칙
  - 높은 우선순위 먼저 설치
  - 선행 관계 준수
  - 한 반복의 점수 합 ≤ 팀 작업 속도

플래닝 포커

- 스토리 점수 합의 방식
- 피보나치 수열 카드
- 순서
  - 카드 준비
  - 기준 스토리 1점 지정
  - 팀원별 점수 추정
  - 동시에 공개
  - 차이가 나면 근거 토론
  - 합의할 때까지 반복

비용 예측

상향식	하향식
소작업별 노력 추정	전체 규모 먼저 추정
기간 × 인력·참여도	과거 데이터로 인력·기간 추정
WBS 기반	LOC·기능점수 기반

노력·인력·기간

- D = E / M
- D: 기간 Duration
- E: 노력 Effort
- M: 투입 인력 비율 합 Manpower
- 핵심 변수: 인력 수 · 작업 기간

예측 기법

- 전문가 판단: 경험자가 WBS 작업별 추정
- PERT: 낙관·가능·비관 기간의 가중 평균
- 알고리즘식 방법: 완료 프로젝트 데이터와 수식 활용

COCOMO-81

- 규모 LOC 기반 수학적 모델
- 노력 = A × Size^B × M
- Size: KDSI · 천 줄 단위 코드량
- M: 15개 노력 승수의 곱
- 한계
  - 초기에 Size 예측 어려움
  - B·M 판단의 주관성
  - 언어별 LOC 차이
  - 재사용 코드 카운트 문제

COCOMO II

- 진행 정도별 모델
  - 프로토타입 단계: UI·3세대 언어 컴포넌트 → 응용 점수
  - 초기 설계 단계: 구조·기능 탐구
  - 구조 설계 이후: 상세 시스템 이해

객체 점수 기반 추정

- 화면·보고서·3세대 언어 컴포넌트 수 계산
- 화면·보고서 복잡도 결정
- 복잡도 가중치 적용
- 객체 점수 OP 계산
- 재사용률 반영
  - NOP = OP × (100 - Reuse) / 100
- 생산성 PROD 결정
- 최종 노력
  - PM = NOP / PROD

기능 점수 FP

- LOC를 모를 때 기능 규모 측정
- 다섯 기능
  - 외부 입력 EI
  - 외부 출력 EO
  - 외부 질의 EQ
  - 내부 논리 파일 ILF
  - 외부 인터페이스 파일 EIF
- 계산
  - 개수 파악

- 복잡도: 단순·중간·복잡
- 가중치 적용
- GFP = Σ(Count × Complexity)
- 14개 처리 복잡도 평가: 0-5점
- PCA = 0.65 + 0.01ΣPC
- FP = GFP × PCA

• 언어별 변환

- 총 LOC = FP × 언어별 1점당 LOC
- 노력 = 총 LOC / 생산성

노션 예시 계산

- GFP = 304
- PCA = 1.11
- FP = 304 × 1.11 = 337.44
- 생산성 60 FP/week
- 노력 E = 337.44 / 60 = 5.624 persons-week

COSMIC 기능점수

- 실시간·인프라 시스템까지 확장
- 데이터 이동 수로 기능 크기 측정
- 순서

- 측정 목적·범위 결정
- 응용층·미들웨어층·DB층 구분
- 트리거·기능 프로세스·데이터 그룹 파악
- Entry · Exit · Read · Write 이동 카운트
- 신규 이동 추가 · 삭제된 이동 제거

프로젝트 팀 조직

- 역할·책임의 위치
- 정보 전달·의사결정 통로
- 갈등 해결 방식
- 주요 역할

- 프로젝트 관리자
- 시스템 운영자·분석가·개발자
- DB 엔지니어
- QA 관리자
- 기술 지원
- 하드웨어·웹·디자인 담당

조직 형태

조직	특징
직능별	부서별 전문 인력 · 단계별 참여 · 협력은 부서 중심
프로젝트별	여러 직능 인력을 한 프로젝트에 배정 · 전달 경로 짧음
매트릭스	직능 조직과 프로젝트의 다대다 결합 · 이중 보고 가능
애자일	5-9명 밀접 협업 · 결과와 이슈의 오너십 공유

- 직능별 → 매트릭스 → 프로젝트별
  - 프로젝트 관리자 권한 증가
  - 동원 가능한 자원 범위 증가
  - 협상·조정 역량 중요

실행과 모니터링

- 실행
  - 작업 시작 미팅
  - 작업 결과 수집
- 모니터링
  - 일정 모니터링
  - 어닝 밸류 분석
  - 번다운 차트

리스크 관리

- 목적: 위험 발생 가능성과 영향 감소
- 절차
  - 리스크 파악
  - 리스크 평가·우선순위
  - 대응 계획
  - 실행 중 추적
- 파악 방법
  - 회의
  - 문서 분석
  - 리스크 분할 구조
  - 체크리스트
  - 유추
- 평가 기준
  - 발생 확률
  - 영향도
- 대응
  - 계획 변경

- 책임 이전
- 프로토타이핑
- 유능한 인재 배치
- 제3자 협업

4장 요구 분석

요구 분석 작업

- 소프트웨어 개발의 실질적 첫 단계
- 사용자 요구 이해·정리
- 세 작업
  - 요구 추출: 인터뷰·설문·자료 조사
  - 요구 분석·정의: 모호성·충돌·우선순위 정리
  - 요구 확인: 프로토타입·검토·테스트 가능성 확인

요구와 제약

- 요구
  - 시스템에 대한 고객의 요청 조건
  - 프로젝트 성공의 필수 요건
  - 여러 이해관계자의 이해가 반영됨
- 제약
  - 특정 프로그래밍 언어
  - 특정 제품·플랫폼
  - 예산·법규·표준
  - 가능한 해결책의 범위 제한

기능 요구와 비기능 요구

기능 요구	비기능 요구
시스템이 해야 할 기능	기능 외에 갖춰야 할 조건·속성
동사 중심	형용사·수치 기준 중심
유즈케이스로 정리	품질 속성 시나리오로 정리
주문·결제·취소	2초 이내 응답·99.9% 가용성·보안

기능 요구 파악 질문

- 시스템은 무엇을 하는가
- 입력·출력은 무엇인가
- 외부 시스템과 주고받는 자료는 무엇인가
- 누가 시스템을 사용하는가
- 특정 상태와 외부 명령에 어떤 반응을 하는가

비기능 요구 종류

- 성능: 처리량 · 응답 시간 · 실시간성 · 자원 이용률
- 품질: 신뢰성 · 가용성 · 오류 발생률

- 안전: 의도하지 않은 동작·상태 방지
- 보안: 악의적 공격으로부터 자원 보호
- 사용성: 인터페이스 · 사용자가 보고 느끼는 경험

대상에 따른 요구

- 비즈니스 요구: 무엇을 해야 하는가
- 아키텍처·설계 요구: 어떻게 동작할 것인가
- 시스템 통합 요구: 어떤 데이터와 연결로 구현할 것인가

요구 추출

- 사용자·출처에서 요구 수집
- 세 단계
  - 정보 출처 파악
  - 정보 취합
  - 요구·제한 사항 정리

정보 출처

- 고객·발주자
- 도메인 전문가
- 이해관계자
- 직접 사용자
- 기존 시스템 역공학
- 업무 문서·요구 템플릿·표준

요구 추출 뷰포인트

- 상호작용 관점
  - 직접 사용자·외부 시스템
  - 행동·입출력·인터페이스 요구
- 간접 관점
  - 관리·보안·감사·유지보수 관계자
- 도메인 관점
  - 업무 영역의 규칙·산업 표준·제약

정보 수집 방법

고객 발표

- 초기 개념 파악
- 업무 책임자가 발표
- 발표 전 필요한 정보 검토
- 구현 논의보다 업무·요구 중심

문헌·업무 조사

- 유사 프로젝트
- 업무 문서·양식·운영 매뉴얼
- 산업·기업 표준
- 정부 정책·규제

설문

- 많은 이해관계자 의견
- 무기명으로 숨은 문제·개선 의견 확보
- 짧고 중요한 질문
- 모호하지 않은 표현

인터뷰

- 다양한 관계자
- 충분한 시간
- 핵심 관계자 반복 인터뷰
- 최대·최소·예외·변동 질문
- 미래 비전
- 최소 허용 솔루션
- 추가 정보원
- 직접 다이어그램 작성 요청

브레인스토밍·JAD

- 아이디어 생성 회의
- 토론보다 아이디어 수집
- 숙련 진행자
- 익명성·상호 자극
- 아이디어 설명 → 분류 → 투표·우선순위

좋은 요구의 여섯 속성

- 원자성: 하나의 요구에 하나의 목적
- 완전성: 필요한 정보 모두 포함
- 비모호성·통일성: 한 가지 뜻 · 용어 일관
- 추적성: 고유 번호 · 출처·설계·테스트 연결
- 우선순위: 중요도·긴급도 구분
- 테스트 가능성: 합격·불합격 판정 가능

도메인 분석

- 도메인: 요구가 속한 업무 배경
- 목적

- 개념·용어 파악
- 비즈니스 규칙 정리
- 설계 모델의 기반 마련

• 세 단계

- 도메인 개념 찾기
- 도메인 사전 작성
- 비즈니스 규칙 정리

시나리오 기반 분석 - 5W1H

• 언제·어디서: 상황
• 누가: 사용자
• 무엇을: 서비스
• 왜: 목표·기대
• 어떻게: 시스템 사용법
• 표현: 문장 · 스토리보드 · 만화 · 음성 · 동영상

사용자 스토리

• 사용자와 개발팀이 함께 작성
• 사용자가 바라는 역량을 짧게 기술
• 형식

- 나는 〔사용자〕로서
- 〔기능〕을 하고 싶다
- 그래서 〔가치〕를 얻는다

• 치킨집 예

- 고객으로서 메뉴별 맵기와 알레르기 정보를 보고 싶다

유즈케이스

• 도메인 분석과 모델링의 관문
• 도메인 개념 → 기능 모델
• 구성

- 액터
- 시스템 범위
- 유즈케이스
- 관계

• 분석 순서

- 액터 찾기
- 유즈케이스 찾기
- 유즈케이스 관계 찾기

액터

- 시스템과 상호작용하는 외부 엔티티
- 사람의 역할
- 다른 시스템·장비도 가능
- 구별되는 이름과 설명 필요
- 찾기 위한 질문
  - 누가 주요 기능을 사용하는가
  - 누가 관리·유지보수 기능을 사용하는가
  - 어떤 외부 시스템·하드웨어와 연결되는가

유즈케이스

- 여러 개별 시나리오의 묶음
- 시스템이 수행하는 일련의 동작
- 특정 액터에게 관찰 가능한 결과 제공
- 정상 흐름 + 오류·예외 흐름
- 개발자와 사용자가 함께 작성

유즈케이스 명세 항목

- 시스템 제목
- 유즈케이스 이름
- 액터
- 시작 조건
- 기본 흐름
- 대안·예외 흐름
- 종료 조건

유즈케이스 관계

관계	의미	화살표 방향
대안 흐름	기본 흐름의 선택·변형·예외	명세 안에서 별도 흐름으로 기술
〈〈include〉〉	공통 동작 재사용 · 중복 제거	기본 유즈케이스 → 포함되는 유즈케이스
〈〈extend〉〉	특정 조건에서 선택적 기능 추가	확장 유즈케이스 → 기본 유즈케이스

- 포함 예
  - 주문하기 → 결제 인증
  - 여러 기능에서 공통 인증 재사용
- 확장 예
  - 멤버십 할인 → 결제
  - 회원 조건 만족 시에만 삽입

요구 명세

- SRS: Software Requirements Specification
- 분석 결과를 표준 문서로 정리
- 설계·구현·테스트의 기준선

요구 검증

- 사용자·고객 목적을 완전히 포함하는가
- 문서 표준을 따르는가
- 설계 단계의 기초로 적합한가
- 내부 내용이 일관되고 완전한가
- 이해관계자의 기대와 일치하는가

5장 요구 모델링

요구 모델링

- 고객과 개발자가 개발 대상을 함께 이해
- 합의 가능한 요구 명세 생성
- 분석과 모델링
  - 분석: 시스템의 형식적·준형식적 설명
  - 모델링: 무엇을 개발할지 시각화·합의
- 영역 구분
  - 문제 영역: 사용자의 업무·문제
  - 솔루션 영역: 소프트웨어 구조·구현 방법

모델링 이유

- 복잡성 관리
- 무형 소프트웨어 구조 시각화
- 사람 사이 의사소통
- 문제 도메인·제품 요구 이해
- 개발 중 시스템 이해
- 구현 전 잠재 솔루션 실험
- 기존 시스템 문서화

관점과 추상화 수준

- 관점
  - 같은 시스템도 보는 목적에 따라 다른 모델
- 추상화 수준
  - 확대: 상세 속성·메서드
  - 축소: 큰 구조·관계
- 지도 비유
  - 위성 지도 · 지형 지도 · 도로 지도

모델 사이 관계

- 도메인 지식
  - 현실 개체와 관계
  - 고객 · 주문 · 상품
- 요구 모델링
  - 시스템이 해야 할 일
  - 사용자 관점
  - 유즈케이스 중심

- 설계 모델링
  - 코드 구조에 가까운 구체화
  - 클래스 책임·인터페이스

UML

- 객체지향 소프트웨어 모델링의 표준 그래픽 언어
- 소프트웨어 모델링의 공통 언어
- 하드웨어의 회로도와 비슷한 역할
- 다이어그램
  - 구조 유형 6개
  - 동작 유형 7개
  - 동작 중 4개는 객체 간 상호작용

UML 모델링 과정

- 요구를 유즈케이스로 정리
- 유즈케이스 다이어그램 작성
- 클래스 후보 추출
- 개념적 클래스 다이어그램 작성
- 유즈케이스별 시퀀스 다이어그램 작성
- 속성·오퍼레이션·관계 추가
- 상태·액티비티 다이어그램 보완
- 서브시스템·아키텍처 설계
- 기존 객체 재사용·커스터마이징·신규 설계

다이어그램 연결

- 시퀀스의 메시지
  - 수신 클래스의 메서드로 정의
- 클래스 다이어그램
  - 상호작용을 구현할 구조
- 점진적 상세화
  - 클래스 이름만 배치
  - 유즈케이스별 시퀀스 작성
  - 메시지에 대응하는 메서드 추가

정적 모델링

- 시간 변화보다 구조 중심
- 객체의 공통 구조·동작 추상화
- 대표: 클래스 다이어그램

객체와 클래스

- 객체
  - 상태
  - 동작
  - 고유 식별자
  - 실제 개별 실체
- 클래스
  - 공통 속성·동작 정의
  - 객체의 설계도
- 관계
  - 클래스 → 인스턴스화 → 객체
- 치킨집 예
  - Order: 클래스
  - 주문번호 101·조리 중: 객체

속성과 오퍼레이션

- 속성 Attribute: 객체의 자료·필드
- 오퍼레이션 Operation: 자료에 적용하는 연산
- 일반 변수 int x: 정수값 저장
- 객체 변수 Order x: 주문 객체 참조 + 오퍼레이션 사용

캡슐화와 정보 은닉

- 캡슐화: 속성 + 오퍼레이션을 하나의 단위로 묶음
- 정보 은닉: 내부 정보 접근 제한
- private: 외부의 직접 접근 차단

객체지향 기본 관계

연관 Association

- 객체 사이 지속적인 구조적 연결
- 다른 객체의 정보를 참조
- 예: 고객 - 주문

집약 Aggregation

- 약한 전체-부분
- 부분이 독립적으로 존재 가능
- 빈 마름모가 전체 쪽
- 예: 치킨집 ◇- 직원

합성 Composition

- 강한 전체-부분

- 부분의 생명주기가 전체에 종속
- 검은 마름모가 전체 쪽
- 예: 주문 ◆- 주문항목

상속 Generalization

- 자식이 부모의 속성·오퍼레이션 상속
- 일반화-구체화 관계
- 자식 객체를 부모 타입 자리에 사용 가능
- 실선 + 빈 삼각형
- 자식 → 부모

다형성 Polymorphism

- 같은 메시지 이름
- 실제 객체 종류에 따라 다른 동작
- pay()
  - CardPayment: 카드 결제
  - BankPayment: 계좌이체

클래스 표현

- 세 구역
  - 클래스 이름
  - 속성
  - 오퍼레이션
- 가시성
  - + public
  - - private
  - # protected
  - ~ package
- 추상 클래스: 이름을 기울임꼴
- 인터페이스: 〈〈interface〉〉
- 흔한 getter·setter: 생략 가능
- 모델링 진행에 따라 상세화

클래스 다이어그램 관계 표기

관계	표기	방향
연관	실선	기본 방향 없음 · 탐색 방향만 일반 화살표
집약	빈 마름모 + 실선	마름모가 전체 쪽
합성	검은 마름모 + 실선	마름모가 전체 쪽
상속	실선 + 빈 삼각형	자식 → 부모

관계	표기	방향
의존	점선 + 일반 화살표	사용하는 클래스 → 사용되는 클래스
구현	점선 + 빈 삼각형	구현체 → 인터페이스

화살표 암기

- 빈 삼각형 → 부모 또는 인터페이스
- 일반 점선 화살표 → 사용하는 쪽에서 사용되는 쪽
- 마름모 → 전체 쪽
- 실선 빈 삼각형 → 상속
- 점선 빈 삼각형 → 구현

다중성 Multiplicity

- 1: 정확히 하나
- 0..1: 없거나 하나
- 0..* 또는 *: 0개 이상
- 1..*: 1개 이상
- 읽는 법
  - 반대편 객체 하나를 기준으로 끝의 수 확인
- 예
  - Customer 1 - 0..* Order
  - 주문 하나 → 고객 한 명
  - 고객 한 명 → 주문 0개 이상

연관 클래스

- 다대다 관계 자체에 정보가 필요한 경우
- 관계를 별도 클래스로 표현
- 예
  - 고객 - 쿠폰
  - CouponUse: 사용 시각·할인 금액

연관과 의존

- 연관: 객체를 구조적으로 알고 있는 관계
- 의존: 기능 수행 중 잠시 사용하는 약한 관계
- 사용되는 클래스 변경 → 사용하는 클래스도 영향 가능

주의

- 이 페이지의 집약·합성은 전체-부분 관계
- 4장의 유즈케이스 〈〈include〉〉와 다른 개념

동적 모델링

- 실행 중 변화·행동 중심
- 정적 모델 보완
- 두 수준
  - 시스템 전체 동작: 유즈케이스
  - 객체·오퍼레이션 상호작용: 시퀀스·커뮤니케이션

시퀀스 다이어그램

- 객체 메시지를 시간 순서로 표현
- 수평축: 참여 객체
- 수직축: 위에서 아래로 시간
- 라이프라인: 객체 생존 구간
- 작성 순서
  - 참여 객체 파악
  - 수평 배치·라이프라인
  - 유즈케이스 이벤트 순서대로 메시지 연결
- 메시지
  - 〈〈create〉〉: 객체 생성
  - 〈〈destroy〉〉: 객체 소멸 · 라이프라인 끝 X
  - 비동기: 응답을 기다리지 않고 다음 작업
- 조건·반복
  - alt: 조건에 따른 선택
  - opt: 조건 만족 시에만 실행
  - loop: 반복

커뮤니케이션 다이어그램

- 객체 사이 링크 + 메시지
- 시간축보다 연결 구조 강조
- 메시지 순서 번호 사용
- 시퀀스와 같은 상호작용을 다른 관점으로 표현

상태 다이어그램

- 한 객체의 상태 변화
- 수신 이벤트·응답·전이
- 상태가 복잡한 클래스 대상
- 전체 시스템 구조 표현용 아님
- 상태 내부 동작
  - entry: 상태 진입 시

- do: 상태 유지 동안
- exit: 상태 이탈 시

- 주문 예
- 접수 → 결제 완료 → 조리 중 → 배달 중 → 완료

제어 모델링 - 액티비티 다이어그램

- 액티비티 사이 제어 흐름
- 업무 절차 중심 흐름도
- 요소
- 액티비티: 계산·프로세스
- 전환: 다음 활동으로 제어 이동
- 분기: 진위 조건
- Fork: 병렬 흐름 분리
- Join: 병렬 흐름 합류
- Swimlane: 담당자별 영역

다이어그램 구분

다이어그램	핵심 질문
클래스	무엇이 있고 어떻게 연결되는가
시퀀스	누가 누구에게 어떤 순서로 메시지를 보내는가
커뮤니케이션	어떤 객체들이 연결되어 메시지를 주고받는가
상태	객체 하나의 상태가 언제 어떻게 바뀌는가
액티비티	업무가 어떤 조건과 순서로 흐르는가

모델 검증 - 리테정프요

- 리뷰
- 워크스루·인스펙션
- 사람이 눈으로 확인

- 테스팅
- 모델에서 테스트 데이터 도출
- 구현 후 실행 결과 검사

- 정형적 방법
- 정확성·일관성을 수학적으로 증명

- 프로토타이핑
- 모델 기반 시제품

- 요구 추적
- 모델 요소에서 원래 요구로 역추적

일관성 체크

유즈케이스 ↔ 시퀀스

- 유즈케이스 명세 존재 여부
- 대응하는 시퀀스 다이어그램 존재 여부
- 기능별 메시지 흐름 누락 여부

시퀀스 ↔ 클래스

- 시퀀스의 클래스가 클래스 다이어그램에 존재
- 메시지가 수신 클래스의 오퍼레이션으로 정의
- 예

– 시퀀스: Payment.pay() 호출

– 클래스: Payment에 pay() 필요

상태 ↔ 클래스

- 상태 전이에 필요한 오퍼레이션 확인
- 메서드 기반 전이라면 해당 클래스에 메서드 정의
- 단, 전이는 외부 이벤트·시간 조건으로도 발생 가능

5장 최종 흐름

- 유즈케이스로 요구 정리
- 클래스로 구조 표현
- 시퀀스로 객체 대화 표현
- 상태로 객체 변화 표현
- 액티비티로 업무 흐름 표현
- 다이어그램 사이 일관성 검증

시험 직전 최종 암기

1장

- 소프트웨어 특징: 복순변비
- 개발 활동: 명세화 · 구현 · 검증 · 유지보수
- 핵심 축: 프로세스 · 품질 보증 · 프로젝트 관리

2장

- 생명주기: 요구 분석 → 설계 → 구현 → 테스트 → 유지보수
- 전통 모델: 폭포수 · V · 프로토타입 · 점증 · 진화 · 나선 · UP
- 애자일: 짧은 주기 · 변화 대응 · 작동 소프트웨어
- 프로세스 = 작업 순서 / 방법론 = 수행 방법

3장

- 계획: 범위 · WBS · 일정 · 비용 · 조직 · 위험
- CPM: 가장 긴 경로가 프로젝트 최소 기간
- TS = TL - TE
- 노력 = A × Size^B × M
- FP = GFP × PCA
- PM = NOP / PROD

4장

- 요구 분석: 추출 · 분석·정의 · 확인
- 좋은 요구: 원자성 · 완전성 · 비모호성 · 추적성 · 우선순위 · 테스트 가능성
- 유즈케이스: 액터 · 시스템 범위 · 기능 · 관계
- include: 공통 기능 / extend: 조건부 기능

5장

- 클래스 = 설계도 / 객체 = 실제 인스턴스
- 삼각형 = 부모·인터페이스 방향
- 마름모 = 전체 쪽
- 의존 = 사용하는 쪽 → 사용되는 쪽
- 시퀀스 = 시간순 메시지
- 상태 = 객체 상태 변화
- 액티비티 = 업무 흐름
- 모델 검증: 리테정프요
- 전체 흐름: 필요한 것 찾기 → 합의 가능한 요구 만들기 → 구조와 행동 모델링 → 일정·비용·인력·위험 안에서 구현 → 테스트와 검증으로 품질 확인